Introduction

The following provides a description of a “Web of Things” inspired, do-it-yourself, wireless weather station perfect for the hobbyist or home investigator. The implementation described below, though certainly not unique, represents a balanced trade off between cost and function. A high priority has been placed on using all "open-source" hardware and software. Avoiding proprietary, closed source software, as well as proprietary “cloud” services, allows for much greater flexibility. This flexibility facilitates ease of deployment and scale-out on a large variety of open source platforms, and for a large variety of end user applications.

For example, the weather station might be in a location which only provides wireless connectivity to the Internet. In this case, the weather station could be configured to send data via the Internet to a remotely located server acting as a client. Alternatively, the weather station might be connected via a local area as a stand alone web server. In this case, the local network router may be configured to allow Internet access to the weather station web page hosted on the server. Many deployment options are possible. This project plan uses the later model to provide an example of a “turn-key” solution that includes both the weather station, and the implementation of “back-end” services.

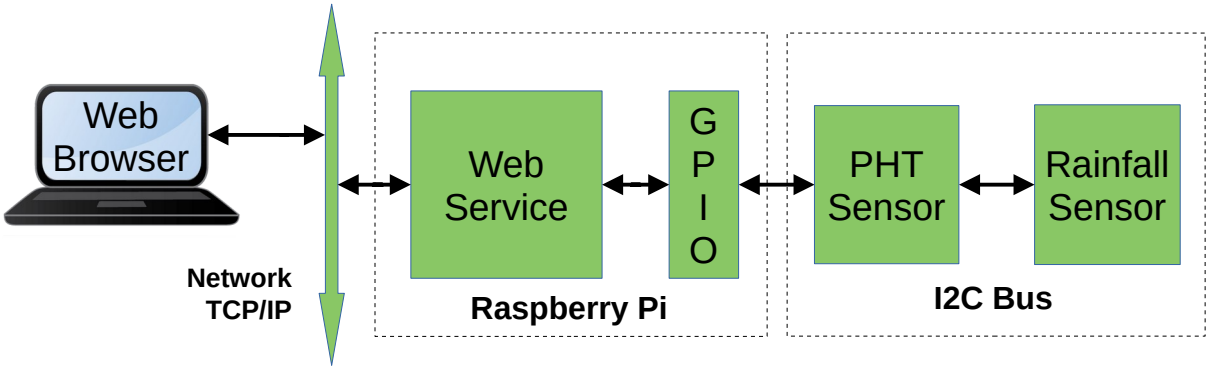


Figure 1. Overall conceptual view showing weather station, network, and server components.

Prerequisites for building and deploying the weather station include: ability to follow electrical hookup guides, knowledge of Raspberry Pi technology, Linux system administration, and some knowledge of the Python computer language. The open source software provided in the Github repository should be sufficient, with only minor modifications, for most applications. The server side software has been designed around the file structure and system configuration found in most common Linux distributions. In most cases, the majority of modification will be in the HTML, customizing the look and appearance of the web page presentation to the end user.

Weather Station Overview

The weather station system consists of three main components. Referring to figure 1, these components are: the Raspberry Pi acting as a micro-controller and HTTP server; the weather sensors which measure pressure, temperature, humidity, and rainfall; and the network connecting the weather station to downstream clients. The following project description will focus on the weather station hardware and software. The network connecting downstream clients, including wireless access points, switches, routers, etc., will be considered background infrastructure beyond the scope of this document.

The server software consists of two main components: an agent for processing data received from the weather station, and HTML documents for displaying the data. The agent component, acting as client, periodically gets the latest weather data from the weather sensors. When running on a mirror server, the agent sends an HTTP request to the weather station, which responds with a short text string containing the latest weather information. The agent, a Python script running as a background process, gets data from the weather sensors, formats and converts the data, writes the data to the rrdtool database, and updates the data output file used by web service. HTML documents get the data from the output file and display the data in user friendly manner on client web browsers.

Component Descriptions

Weather Station Hardware

Referring to figure 1, the weather station hardware consists of a Raspberry Pi, a PTH sensor, and a rainfall sensor. The PTH sensor measures barometric pressure, ambient temperature, and relative humidity. The rainfall sensor measures rainfall using the bucket gauge method. Both the the PTH sensor and the rainfall sensor are connected to the Raspberry Pi's GPIO I2C serial data I/O (see schematic). An external bucket rain gauge is connected to the rainfall sensor.

Weather Station Software

Referring to figure 2, the weather station software consists essentially of two components: a web server component and an agent component. The web server component includes PHP scripts and HTML documents that contain Javascript for displaying charts and weather information. The agent, a Python script, manages data conversion and reformatting, updating a database, and creating charts.

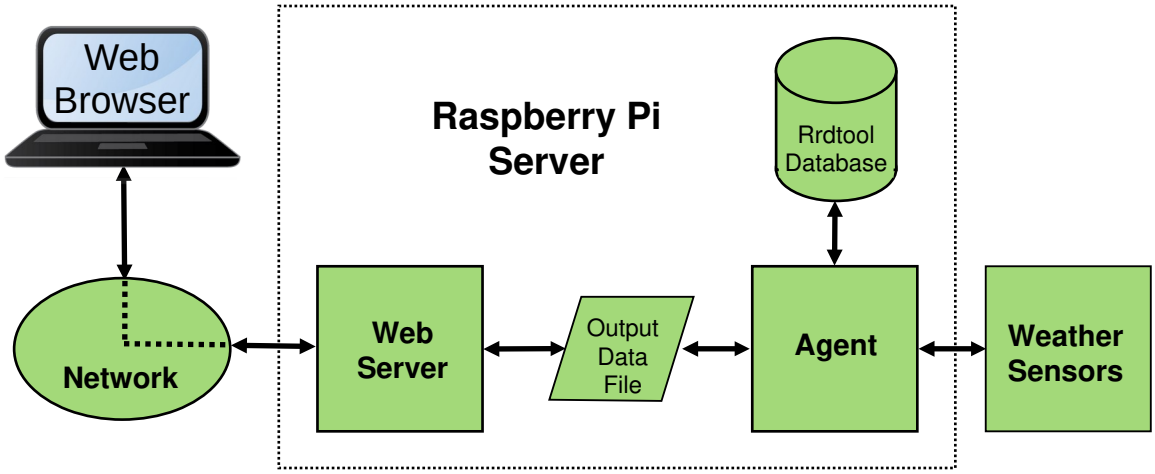


Figure 2. Block diagram of weather station software components showing weather sensors, agent process, web server, and database elements.

The agent periodically polls the weather sensors for pressure, temperature, humidity, and rainfall. The agent formats the sensor data and converts the data to other physical units where required. Selected data items also get written to a round-robin database for permanent storage. Besides these functions, the agent manages generation of graphic charts for display in HTML documents.

After formatting the weather data, the agent writes the data to the *Output Data File* for use by HTML documents. When a client browser requests the weather HTML document, Javascript embedded in the document periodically requests the output data file and displays the data in an web page. Although not shown in figure 2, the agent, as mentioned earlier, generates graphic charts for display in the HTML documents. The graphic charts are stored as image files which Javascript in the HTML documents downloads and displays in the weather web page.

Referring to figure 3, the agent consists of several software modules. The main module, *weatherAgent.py* handles all high level functions: polling weather sensors, formatting and converting the data from the sensors, writing data to the round-robin database, writing data to a JSON file for use by the web server, and generation of charts for display in the weather web page.

The agent imports three modules which act as an interface to sensor devices and to the round-robin database. The *rainfall* module aggregates raw rainfall bucket tips into daily rainfall and hourly rainfall. The *rainfall* module imports the *sen0575* module, which gets raw bucket tips from the SEN0575 sensor. The *ms8607* module gets raw pressure, temperature, and humidity data as binary values and then converts these binary value to floating point numbers in units of pascals, degrees Celsius, and percent relative humidity. The *rrdbase* module acts as an interface to the round-robin database, providing methods that execute **rrdtool** commands for updating the database and generating charts.

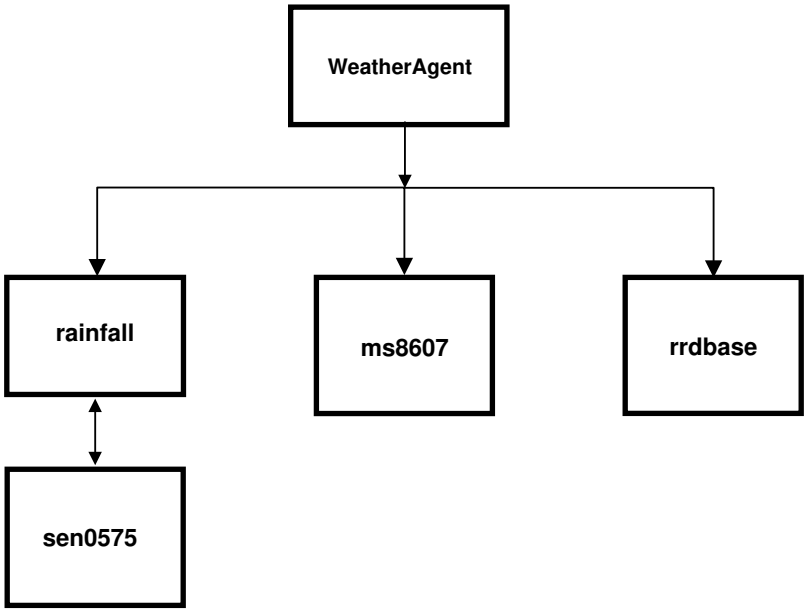


Figure 3. Weather station software block diagram showing high level components.

The flow of events run by the weather agent may be summarized as follows:

Initial setup, run once when the agent starts.

1. Print a startup message.
2. Get command line arguments and parameters.

3. Check for the existence of the weather round-robin database. Exit with error code if it does not exist.
4. Determine whether we are running as source server or a mirror server. A source server gets data directly from the weather sensors, while a mirror server gets data from a source server. The mirror server option allows for using a much more robust infrastructure that can handle large user loads.
5. If we are a source server, instantiate sensor objects from the *ms8607* class and the *rainfall* class. The classes are imported from their respective modules. Instantiating the *rainfall* object in turn instantiates a *sen0575* object. These objects provide methods for getting data from the sensors.
6. Finally, instantiate a object for accessing the database from the *rrdbase* class, imported from the *rrdbase* module. The *rrdbase* object provides methods for updating the round-robin database, and for creating charts from data stored in the database.

Main loop, run continuously after initial setup.

1. If the wait time between data requests has elapsed, go to step 2.
2. If we are a mirror server, go to step 6. Otherwise we are a source server, so do steps 3 thru 5.
3. If midnight (00:00 hrs), reset the daily rainfall total to zero.
4. Convert the data from the sensors to appropriate units.
5. Go to step 7.
6. Send an HTTP request to the source server for the weather data.
7. Write the weather data to a Javascript Object Notation (JSON) file accessible to to the Apache2 server.
8. If the wait time between round-robin database updates has elapsed, update the database with the latest weather data.
9. If the wait time between day chart creations has elapsed, launch a process to create new day charts.
10. If the wait time between long term chart creations has elapsed, launch a process to create new long term charts.
11. Return to step 1.

Building the Weather Station

Parts List

To build the weather station, see the list below for recommended parts and materials. Consult the supplier's web site for detailed instructions on how to hookup and use individual parts.

Description	Quantity	Supplier	Part #
Raspberry Pi Zero W	1	Sparkfun	DEV-15470
MS8607 PTH Sensor (discontinued)	1	Sparkfun	PRT-12081
SEN0575 DF Robot Rainfall Sensor	1	DF Robot	SEN0575
Flexible Qwiic Cable 50mm	1	Sparkfun	PRT-17260
Qwiic to Gravity Cable	1	Sparkfun	CAB-28768

Description	Quantity	Supplier	Part #
USB A to Micro USB Power Cable		Amazon	Product Link
Solar Radiation Shield (housing)	1	Ambient	SRS100LX

Note: the MS8607 sensor has been discontinued and is no longer available. A suitable replacement would be the BME280 sensor (Sparkfun # SEN-15440). Of course, the MS8607 module would need to be rewritten for the BME280.

Supplier Websites
Ordering information, detailed specifications, and hookup guides are available online at

- Amazon: amazon.com
- Ambient LLC: ambientweather.com
- DF Robot: dfrobot.com
- Sparkfun: sparkfun.com

Some additional hardware will be required, such as: a plastic card on which to mount the Raspberry Pi and attached weather sensors, mounting hardware, etc.

Assembly Notes

1. Connect the MS8607 sensor to the Raspberry Pi GPIO pins as shown on the Weather Station Schematic. Use a Qwiic male connector to female jumper (4-pin) cable to connect the sensor to the Raspberry Pi.
2. Connect the SEN0575 sensor to the MS8607 sensor using a male Qwiic connector to male Gravity connector cable.
3. The Pi Zero and the sensors should be mounted on a plastic card that can easily be inserted into Ambient Solar Radiation Shield. Installing the Pi Zero in a case made for the Pi Zero before mounting to the plastic card will help prevent moisture and dust from getting into the Pi Zero.
4. Connect the rain bucket gauge RJ11 connector to the SEN0575 sensor.
5. Connect a USB to micro-USB power cable to the Pi Zero.
6. At this point the Pi Zero should have Apache2 configured and the weather station software installed. See instructions below for configuring the Pi Zero and installing the software.
7. Carefully insert the entire assembly into the solar radiation shield. Surrounding the assembly with a flexible, plastic screen can help keep insects away from circuit boards and out of the electronics.
8. Mount the solar radiation shield on the supplied bracket in an appropriate place outdoors with free flowing air.
9. The Raspberry Pi requires an external 5 volt power supply. The power supply should have an outdoor rated cable with the correct USB connector for the Raspberry Pi. The power supply should be plugged into an outdoor rated electrical outlet.

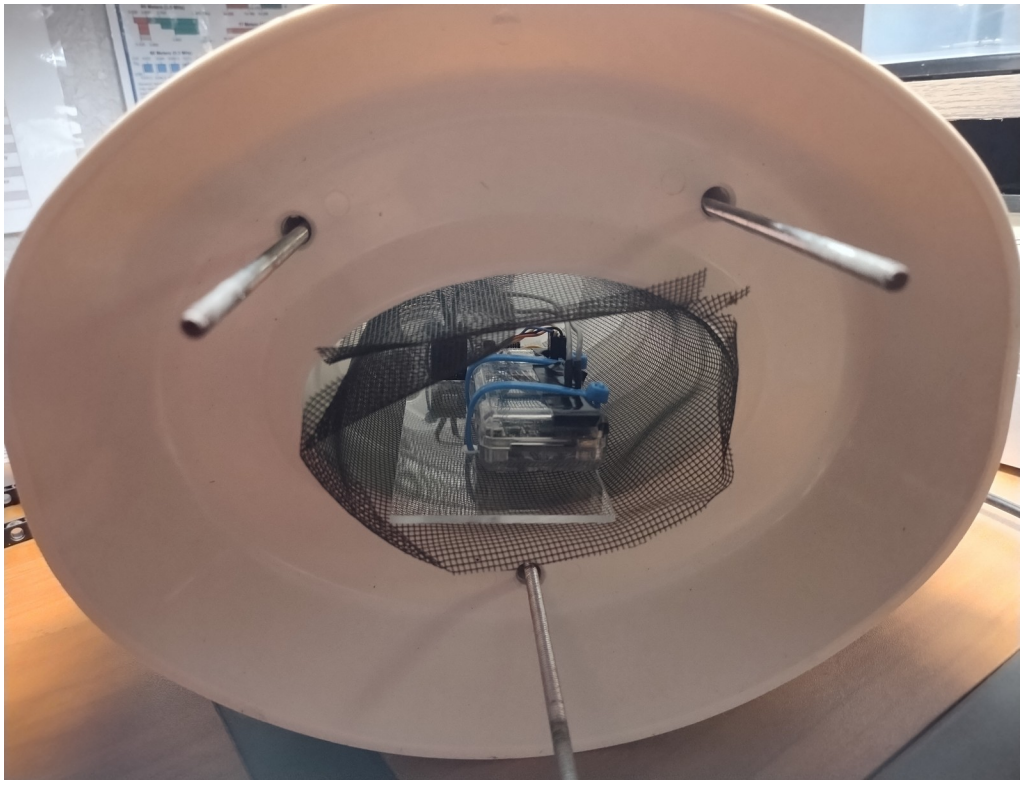


Figure n. Pi Zero and sensors mounted on a plastic card which is inserted into Ambient Solar Radiation Shield. Note the screening use to keep insects out.

Installing the Weather Station Software

Dependencies

• The weather station software should be installed on a recent version of Raspbian. If running as a mirror server the sofware can be installed on a recent version of almost any Linux distribution, such as Ubuntu.
• Apache2 should be installed and configured to allow serving HTML documents from the user's public_html folder.
• Rrdtool should be installed.
• Python 3 usually comes installed in virtually all current Linux distributions. Type "python" at a command line prompt to verify Python has been installed.

Software Inventory

The the following software items that need to be installed.

From the repository html folder:

- weather.html
- weather.php
- index.html
- static/chalk.jpg

From the repository bin folder:

- createWeatherRrd.py
- weatherAgent.py
- ms8607.py
- sen0575.py
- rainfall.py
- rrdbase.py
- weastart
- weastop

The above software items may be found in their respective folders in the project repository.

Software Installation on a Raspberry Pi Zero W

Note that the following installation procedure assumes that weather HTML documents will reside in the user's public_html directory. Typically the full path name to this directory will be something like "/home/[user_name]/public_html". Although the following steps are specifically for the Raspberry Pi, by using similar steps the software can be installed on almost any Linux distribution.

1. Follow the instructions in the "Base System Installation Guide" to setup and configure apache2 web server on a Raspberry Pi.

2. Navigate to the /**weather-master**/**Raspberry Pi Model**/ folder. The following steps will involve copying files from this folder. You will need to use a file copy method such as secure file copy (**scp**) to copy from your computer to the Raspberry Pi.

3. In the user **public_html** folder, use **mkdir** to create a folder named **weather** that will contain the weather HTML and PHP files. The full path should look like

/home/pi/public_html/weather/.

4. Copy all the contents of the repository **/weather-master/Raspberry Pi Model//html** folder to the **weather** folder created in step 3.

5. In the **weather** folder created in step 3, create a symbolic link to **/tmp/weather** in the temporary file system by running

   ln -s /tmp/weather dynamic

6. If it doesn't already exist, use **mkdir** to create a folder named **bin** in the **pi** user's home folder. The full path should look like **/home/pi/bin**.

7. Copy all the contents of the repository **/weather-master/Raspberry Pi Model//bin/** folder to the folder created in step 6.

8. Make sure all the files copied to the **bin** folder in step 7 have execute permissions. For example, by navigating to the **bin** folder and running

   chmod +x *

9. Create a folder in the temporary file system for dynamic content, and modify the folder's ownership and permissions to allow the Apache www-data user to have access.

   mkdir /tmp/weather
   sudo chown :www-data /tmp/weather
   chmod g+w /tmp/weather

10. To run the above commands whenever the Raspberry Pi boots, create a start up script in the user bin folder.

    touch /home/pi/bin/startup.sh

    Place the commands in step 9 in **startup.sh.**

11. Be sure to give the start up script execute permissions by running

    chmod +x /home/pi/bin/startup.sh

12. If it doesn't already exist, create the file **/etc/rc.local**. Set the permissions to allow **rc.local** execution permissions. E.g.

    sudo touch /etc/rc.local
    sudo chmod 750 /etc/rc.local

    Add to **/etc/rc.local** the following lines

    #!/bin/sh -e
    (su - pi -c "bin/startup.sh")&
    exit 0

    If **/etc/rc.local** already exists, back up **/etc/rc.local** and add the line

(su - pi -c "bin/startup.sh")&

Be sure **exit 0** is the last line of the script. Whenever the host boots up, the **startup.sh** script will run the start up commands.

13. If it doesn't already exist, use **mkdir** to create a folder named **log** in the **pi** user's home folder. The full path should look like /**home**/**pi**/**log**.

14. In the user home folder, create a folder named **database**. The full path to this folder should look like /**home**/**pi**/**database**.

15. In the user **bin** folder run the python script **createWeatherRrd.py**. Running this script creates an empty round robin database file where the agent will store the radiation data. This script should be run once and then kept in a secure place. Running it accidentally at some future date will result in *total loss of all previously stored data*. At the command prompt enter

    ~/**bin**/**createWeatherRrd.py**

16. In most Linux installations the user's bash profile will automatically add the user's bin folder to the command search path. If such is the case, then the agent can be started up by simply typing **weatherAgent.py** followed by ENTER. Otherwise the agent can be started by typing ~/**bin**/**weatherAgent.py**. Note that you will need to edit **weatherAgent.py**, changing the IP address in the default radiation monitor URL to that of your radiation monitor.

17. For convenience two scripts have been provided to make it easy to turn the agent on and off. The **weastart** script starts up the agent and causes all diagnostic output and error messages to be written to a log file in the log folder. The **weastop** script stops the agent from running. The **weastart** command can be placed in the **startup.sh** script mentioned in step 9 to automatically start the agent when the host boots up.

18. In the **weastart** script look for a lines like the following

    SOURCE_URL="{URL of source server if this is a mirror server}"
    SOURCE_SERVER="{your weather station host name}"

    If you are setting up a mirror server replace "{URL of source server if this is a mirror server}" with the URL of your weather station. Replace "{your weather station host name}" with the host name of your weather station.

19. To start the agent type /**home**/**pi**/**bin**/**weastart** at the command line prompt.

20. Append **weather**/ to the Raspberry Pi home page URL and open the URL in a browser running on a computer connected to the same network as the Pi. The URL should look something like **http://raspberrypiHostname.local/weather**/. If all went well you should see a web page showing the current radiation measurement and charts of previous measurements.

This completes installation of the server software.

References and Resources

The following sources should be consulted before building the weather station:

- Raspberry Pi - www.raspberrypi.com/products/raspberry-pi-zero-2-w/

The following tutorials are useful for more in depth understanding of the server software:

- Java Script - www.w3schools.com/js/default.asp
- PHP - www.w3schools.com/php/default.asp
- HTML - www.w3schools.com/html/default.asp
- Rrdtool - oss.oetiker.ch/rrdtool/
- Python - greenteapress.com/thinkpython/thinkpython.html